

MySQL Server Health

Check_mysql_health.sh

```
#!/bin/bash

# =====
# MySQL Health Check Report
# =====

# ----- CONFIGURATION -----

MYSQL_USER="monitor_user"
MYSQL_PASSWORD="YOUR_PASSWORD"

MYSQL_HOST="127.0.0.1"
MYSQL_PORT="3306"

CPU_THRESHOLD=90
DISK_THRESHOLD=85
LONG_QUERY_SECONDS=300
CONNECTION_THRESHOLD=80

REPORT_DIR="/var/log/mysql_health"
REPORT_FILE="${REPORT_DIR}/mysql_health_$(date +%Y-%m-%d_%H-%M-%S).txt"

# Change this according to your backup location
BACKUP_DIR="/backup"

# ----- CREATE REPORT DIRECTORY -----

mkdir -p "$REPORT_DIR"

# ----- COLORS -----

GREEN="\033[0;32m"
YELLOW="\033[1;33m"
RED="\033[0;31m"
NC="\033[0m"

# =====
# FUNCTIONS
# =====

PASS_COUNT=0
WARNING_COUNT=0
CRITICAL_COUNT=0

pass()
{
    echo "[PASS] $1" | tee -a "$REPORT_FILE"
    PASS_COUNT=$((PASS_COUNT+1))
}
```

```

warning()
{
    echo "[WARNING] $1" | tee -a "$REPORT_FILE"
    WARNING_COUNT=$((WARNING_COUNT+1))
}

critical()
{
    echo "[CRITICAL] $1" | tee -a "$REPORT_FILE"
    CRITICAL_COUNT=$((CRITICAL_COUNT+1))
}

section()
{
    echo "" | tee -a "$REPORT_FILE"
    echo "===== " | tee -a
"$REPORT_FILE"
    echo "$1" | tee -a "$REPORT_FILE"
    echo "===== " | tee -a
"$REPORT_FILE"
}

# =====
# REPORT HEADER
# =====

echo "===== " | tee "$REPORT_FILE"
echo "      MYSQL DAILY HEALTH CHECK" | tee -a "$REPORT_FILE"
echo "===== " | tee -a
"$REPORT_FILE"

echo "Server    : $(hostname)" | tee -a "$REPORT_FILE"
echo "Date      : $(date)" | tee -a "$REPORT_FILE"
echo "IP Address  : $(hostname -I | awk '{print $1}')" | tee -a "$REPORT_FILE"

# =====
# 1. MYSQL SERVICE
# =====

section "1. MYSQL SERVICE"

if systemctl is-active --quiet mysql; then
    pass "MySQL service is RUNNING"
else
    critical "MySQL service is DOWN"
fi

# =====
# 2. MYSQL CONNECTION
# =====

section "2. MYSQL CONNECTION"

```

```
MYSQL_CMD="mysql -h${MYSQL_HOST} -P${MYSQL_PORT} -u${MYSQL_USER} -  
p${MYSQL_PASSWORD} -N -B"
```

```
if $MYSQL_CMD -e "SELECT 1;" >/dev/null 2>&1; then  
    pass "MySQL connection successful"  
else  
    critical "Unable to connect to MySQL"  
fi
```

```
# =====  
# 3. CPU  
# =====
```

```
section "3. CPU"
```

```
CPU_USAGE=$(top -bn1 | grep "Cpu(s)" | awk '{print 100 - $8}' | cut -d. -f1)
```

```
echo "CPU Usage   : ${CPU_USAGE}%" | tee -a "$REPORT_FILE"  
echo "Threshold   : ${CPU_THRESHOLD}%" | tee -a "$REPORT_FILE"
```

```
if [ "$CPU_USAGE" -ge "$CPU_THRESHOLD" ]; then  
    critical "CPU usage is HIGH"  
else  
    pass "CPU usage is normal"  
fi
```

```
# =====  
# 4. MEMORY  
# =====
```

```
section "4. MEMORY"
```

```
free -h | tee -a "$REPORT_FILE"
```

```
MEMORY_USAGE=$(free | awk '/Mem:/ {printf "%.0f", ($3/$2)*100}')
```

```
echo "Memory Usage : ${MEMORY_USAGE}%" | tee -a "$REPORT_FILE"
```

```
if [ "$MEMORY_USAGE" -ge 90 ]; then  
    critical "Memory usage is HIGH"  
elif [ "$MEMORY_USAGE" -ge 80 ]; then  
    warning "Memory usage is elevated"  
else  
    pass "Memory usage is normal"  
fi
```

```
# =====  
# 5. DISK  
# =====
```

```
section "5. DISK"
```

```
df -h | tee -a "$REPORT_FILE"
```

```

DISK_USAGE=$(df -P "$BACKUP_DIR" | awk 'NR==2 {gsub("%", "", $5); print $5}')

echo "Backup Disk Usage : ${DISK_USAGE}%" | tee -a "$REPORT_FILE"
echo "Threshold      : ${DISK_THRESHOLD}%" | tee -a "$REPORT_FILE"

if [ "$DISK_USAGE" -ge "$DISK_THRESHOLD" ]; then
    critical "Disk usage is HIGH"
else
    pass "Disk usage is normal"
fi

# =====
# 6. CONNECTIONS
# =====

section "6. MYSQL CONNECTIONS"

CURRENT_CONNECTIONS=$(($MYSQL_CMD -e \
"SHOW GLOBAL STATUS LIKE 'Threads_connected';" | awk '{print $2}')

MAX_CONNECTIONS=$(($MYSQL_CMD -e \
"SHOW VARIABLES LIKE 'max_connections';" | awk '{print $2}')

CONNECTION_PERCENT=$((CURRENT_CONNECTIONS * 100 / MAX_CONNECTIONS))

echo "Current Connections : $CURRENT_CONNECTIONS" | tee -a "$REPORT_FILE"
echo "Max Connections    : $MAX_CONNECTIONS" | tee -a "$REPORT_FILE"
echo "Usage              : ${CONNECTION_PERCENT}%" | tee -a "$REPORT_FILE"

if [ "$CONNECTION_PERCENT" -ge 90 ]; then
    critical "Too many MySQL connections"
elif [ "$CONNECTION_PERCENT" -ge "$CONNECTION_THRESHOLD" ]; then
    warning "MySQL connection usage is HIGH"
else
    pass "MySQL connections are normal"
fi

# =====
# 7. LONG RUNNING QUERIES
# =====

section "7. LONG RUNNING QUERIES"

LONG_QUERIES=$(($MYSQL_CMD -e "
SELECT COUNT(*)
FROM information_schema.PROCESSLIST
WHERE COMMAND <> 'Sleep'
AND TIME >= ${LONG_QUERY_SECONDS};
")

echo "Queries running >= ${LONG_QUERY_SECONDS} seconds : $LONG_QUERIES" | tee -a
"$REPORT_FILE"

if [ "$LONG_QUERIES" -gt 0 ]; then

```

```

warning "Long-running queries detected"

echo "" | tee -a "$REPORT_FILE"
echo "Details:" | tee -a "$REPORT_FILE"

$MYSQL_CMD -e "
SELECT ID, USER, HOST, DB, TIME, STATE, LEFT(INFO,100)
FROM information_schema.PROCESSLIST
WHERE COMMAND <> 'Sleep'
AND TIME >= ${LONG_QUERY_SECONDS}
ORDER BY TIME DESC;
" | tee -a "$REPORT_FILE"

else
    pass "No long-running queries detected"
fi

# =====
# 8. DEADLOCKS
# =====

section "8. DEADLOCKS"

DEADLOCKS=$(($MYSQL_CMD -e "
SHOW GLOBAL STATUS LIKE 'Innodb_deadlocks';
" | awk '{print $2}'))

echo "Total Deadlocks : $DEADLOCKS" | tee -a "$REPORT_FILE"

if [ "$DEADLOCKS" -gt 0 ]; then
    warning "Deadlocks have been detected"
else
    pass "No deadlocks detected"
fi

# =====
# 9. BACKUP
# =====

section "9. BACKUP"

if [ -d "$BACKUP_DIR" ]; then

    LAST_BACKUP=$(find "$BACKUP_DIR" -type f -printf '%T@ %p\n' 2>/dev/null \
    | sort -n | tail -1 | cut -d' ' -f2-)

    if [ -n "$LAST_BACKUP" ]; then

        echo "Last Backup : $LAST_BACKUP" | tee -a "$REPORT_FILE"

        BACKUP_TIME=$(stat -c %Y "$LAST_BACKUP")
        CURRENT_TIME=$(date +%s)

        BACKUP_AGE=$((CURRENT_TIME - BACKUP_TIME))
    fi
fi

```

```

BACKUP_AGE_HOURS=$((BACKUP_AGE / 3600))

echo "Backup Age : ${BACKUP_AGE_HOURS} hours" | tee -a "$REPORT_FILE"

if [ "$BACKUP_AGE_HOURS" -gt 24 ]; then
    critical "Backup is older than 24 hours"
else
    pass "Recent backup found"
fi

else
    critical "No backup found"
fi

else
    critical "Backup directory does not exist: $BACKUP_DIR"
fi

# =====
# 10. FINAL SUMMARY
# =====

section "FINAL SUMMARY"

echo "PASS : $PASS_COUNT" | tee -a "$REPORT_FILE"
echo "WARNING : $WARNING_COUNT" | tee -a "$REPORT_FILE"
echo "CRITICAL : $CRITICAL_COUNT" | tee -a "$REPORT_FILE"

echo "" | tee -a "$REPORT_FILE"

if [ "$CRITICAL_COUNT" -gt 0 ]; then
    echo "OVERALL STATUS : CRITICAL" | tee -a "$REPORT_FILE"
elif [ "$WARNING_COUNT" -gt 0 ]; then
    echo "OVERALL STATUS : WARNING" | tee -a "$REPORT_FILE"
else
    echo "OVERALL STATUS : HEALTHY" | tee -a "$REPORT_FILE"
fi

echo "" | tee -a "$REPORT_FILE"
echo "Report saved at:" | tee -a "$REPORT_FILE"
echo "$REPORT_FILE" | tee -a "$REPORT_FILE"

echo "===== " | tee -a
"$REPORT_FILE"

Setup :
vi /usr/local/bin/check_mysql_health.sh
chmod +x /usr/local/bin/check_mysql_health.sh
Test : /usr/local/bin/check_mysql_health.sh

Import : cat /var/log/mysql_health/mysql_health_*.txt

```

sudo mkdir -p /usr/local/mysql-monitor/{reports,logs}

```
root@db-03:/#  
root@db-03:/# sudo mkdir -p /usr/local/mysql-monitor/{reports,logs}
```

File Archetecture :

```
/usr/local/mysql-monitor/  
|  
├─ config.conf  
├─ check_mysql_health.sh  
├─ check_replication.sh  
├─ check_disk.sh  
├─ backup_mysql.sh  
|  
├─ reports/  
|   └─ 2026-09-03/  
|  
└─ logs/  
    ├── health.log  
    ├── replication.log  
    ├── disk.log  
    └─ backup.log
```

Create a direcotory :

sudo mkdir -p /usr/local/mysql-monitor/{reports,logs}

ls -l /usr/local/mysql-monitor/

Expected :
config.conf
check_mysql_health.sh
check_replication.sh
check_disk.sh
backup_mysql.sh
reports/
logs/

Central configuration file :

sudo vi /usr/local/mysql-monitor/config.conf

```
root@db-03:/#  
root@db-03:/# sudo vi /usr/local/mysql-monitor/config.conf
```

```
MYSQL_HOST="127.0.0.1"
MYSQL_PORT="3306"
MYSQL_USER="monitor_user"
```

```
CPU_THRESHOLD=90
DISK_THRESHOLD=85
MEMORY_THRESHOLD=90
```

```
CONNECTION_WARNING=80
CONNECTION_CRITICAL=90
```

```
LONG_QUERY_SECONDS=300
REPLICATION_LAG_WARNING=300
REPLICATION_LAG_CRITICAL=600
```

```
BACKUP_MAX_AGE_HOURS=24
```

```
BACKUP_DIR="/backup/mysql"
```

```
BASE_DIR="/usr/local/mysql-monitor"
REPORT_DIR="${BASE_DIR}/reports"
LOG_DIR="${BASE_DIR}/logs"
```

Note : don't put password here above script .

Use : .my.cnf

Create monitoring user :

```
CREATE USER 'monitor_user'@'localhost'
IDENTIFIED BY 'StrongMonitoringPassword';
```

```
GRANT PROCESS, REPLICATION CLIENT
ON *.*
TO 'monitor_user'@'localhost';
```

```
GRANT SELECT
ON performance_schema.*
TO 'monitor_user'@'localhost';
```

```
FLUSH PRIVILEGES;
```

```
vi ~/.my.cnf
[client]
user=monitor_user
password=StrongMonitoringPassword
host=127.0.0.1
port=3306
```



```
[client]
user=monitor_user
password=StrongMonitoringPassword
host=127.0.0.1
port=3306
```

chmod 600 ~/.my.cnf

Test : mysql -e "SELECT VERSION();"

```
root@db-03:~# mysql -e "SELECT VERSION();"
+-----+
| VERSION() |
+-----+
| 8.0.44-0ubuntu0.22.04.1 |
+-----+
```

Disk check script

sudo vi /usr/local/mysql-monitor/check_disk.sh

```
#!/bin/bash
```

```
source /usr/local/mysql-monitor/config.conf
```

```
REPORT="{REPORT_DIR}/${date +%Y-%m-%d}"
mkdir -p "$REPORT"
```

```
OUTPUT="{REPORT}/disk_report.txt"
```

```
echo "===== " > "$OUTPUT"
echo "MYSQL DISK HEALTH CHECK" >> "$OUTPUT"
echo "Date: $(date)" >> "$OUTPUT"
echo "===== " >> "$OUTPUT"
```

```
df -h >> "$OUTPUT"
```

```
echo "" >> "$OUTPUT"
echo "Disk usage above ${DISK_THRESHOLD}%:" >> "$OUTPUT"
```

```
df -P | awk -v threshold="${DISK_THRESHOLD}" '
NR > 1 {
    gsub("%", "", $5)
    if ($5 >= threshold)
        print $0
}' >> "$OUTPUT"
```

```
echo "" >> "$OUTPUT"
echo "Report: $OUTPUT"
```

Permission : sudo chmod +x /usr/local/mysql-monitor/check_disk.sh

Run : /usr/local/mysql-monitor/check_disk.sh

Replication check

sudo vi /usr/local/mysql-monitor/check_replication.sh

```

#!/bin/bash

source /usr/local/mysql-monitor/config.conf

REPORT="{REPORT_DIR}/${date +%Y-%m-%d}"
mkdir -p "$REPORT"

OUTPUT="{REPORT}/replication_report.txt"

echo "=====" > "$OUTPUT"
echo "MYSQL REPLICATION HEALTH CHECK" >> "$OUTPUT"
echo "Date: $(date)" >> "$OUTPUT"
echo "=====" >> "$OUTPUT"

mysql -e "SHOW REPLICA STATUS\G" >> "$OUTPUT" 2>&1

echo "" >> "$OUTPUT"

IO_RUNNING=$(mysql -N -e \
"SELECT IFNULL((SELECT SERVICE_STATE FROM performance_schema.replication_connection_status
LIMIT 1),'UNKNOWN');")

SQL_RUNNING=$(mysql -N -e \
"SELECT IFNULL((SELECT SERVICE_STATE FROM performance_schema.replication_applier_status
LIMIT 1),'UNKNOWN');")

echo "IO/Connection State : $IO_RUNNING" >> "$OUTPUT"
echo "SQL/Applier State : $SQL_RUNNING" >> "$OUTPUT"

if [[ "$IO_RUNNING" == "ON" && "$SQL_RUNNING" == "ON" ]]; then
    echo "STATUS : PASS" >> "$OUTPUT"
else
    echo "STATUS : WARNING" >> "$OUTPUT"
fi

echo "Report: $OUTPUT"

```

Backup script

```

sudo vi /usr/local/mysql-monitor/backup_mysql.sh

#!/bin/bash

source /usr/local/mysql-monitor/config.conf

DATE=$(date +%Y-%m-%d_%H-%M-%S)

BACKUP_PATH="{BACKUP_DIR}/${DATE}"
LOG="{LOG_DIR}/backup.log"

mkdir -p "$BACKUP_PATH"

echo "=====" >> "$LOG"
echo "Backup started: $(date)" >> "$LOG"

mysqldump \

```

```

--single-transaction \
--routines \
--triggers \
--events \
--all-databases \
> "${BACKUP_PATH}/mysql_full_${DATE}.sql" 2>> "$LOG"

if [ $? -eq 0 ]; then

    echo "BACKUP_STATUS=SUCCESS" > "${BACKUP_PATH}/backup.status"

    echo "Backup SUCCESS: $(date)" >> "$LOG"

else

    echo "BACKUP_STATUS=FAILED" > "${BACKUP_PATH}/backup.status"

    echo "Backup FAILED: $(date)" >> "$LOG"

fi

echo "Backup finished: $(date)" >> "$LOG"

Permission : sudo chmod +x /usr/local/mysql-monitor/backup_mysql.sh

Main health check
sudo vi /usr/local/mysql-monitor/check_mysql_health.sh

#!/bin/bash

source /usr/local/mysql-monitor/config.conf

DATE=$(date +%Y-%m-%d)
REPORT="${REPORT_DIR}/${DATE}"

mkdir -p "$REPORT"

OUTPUT="${REPORT}/mysql_health_report.txt"

PASS=0
WARNING=0
CRITICAL=0

echo "===== " > "$OUTPUT"
echo "      MYSQL HEALTH REPORT" >> "$OUTPUT"
echo "===== " >> "$OUTPUT"

echo "Server : $(hostname)" >> "$OUTPUT"
echo "Date   : $(date)" >> "$OUTPUT"

# -----
# MYSQL SERVICE

```

```

# -----

echo "" >> "$OUTPUT"
echo "1. MYSQL SERVICE" >> "$OUTPUT"
echo "-----" >> "$OUTPUT"

if systemctl is-active --quiet mysql; then

    echo "  PASS : MySQL service is running" >> "$OUTPUT"
    PASS=$((PASS+1))

else

    echo "  CRITICAL : MySQL service is DOWN" >> "$OUTPUT"
    CRITICAL=$((CRITICAL+1))

fi

# -----
# MYSQL CONNECTION
# -----

echo "" >> "$OUTPUT"
echo "2. MYSQL CONNECTION" >> "$OUTPUT"
echo "-----" >> "$OUTPUT"

if mysql -e "SELECT 1;" >/dev/null 2>&1; then

    echo "  PASS : MySQL connection successful" >> "$OUTPUT"
    PASS=$((PASS+1))

else

    echo "  CRITICAL : Cannot connect to MySQL" >> "$OUTPUT"
    CRITICAL=$((CRITICAL+1))

fi

# -----
# CPU
# -----

echo "" >> "$OUTPUT"
echo "3. CPU" >> "$OUTPUT"
echo "-----" >> "$OUTPUT"

CPU=$(top -bn1 | awk '/Cpu\(\s\)/ {print 100 - $8}' | cut -d. -f1)

echo "CPU Usage : ${CPU}%" >> "$OUTPUT"

if [ "$CPU" -ge "$CPU_THRESHOLD" ]; then

    echo "  CRITICAL : CPU > ${CPU_THRESHOLD}%" >> "$OUTPUT"
    CRITICAL=$((CRITICAL+1))

```

```

else

    echo " PASS : CPU normal" >> "$OUTPUT"
    PASS=$((PASS+1))

fi

# -----
# MEMORY
# -----

echo "" >> "$OUTPUT"
echo "4. MEMORY" >> "$OUTPUT"
echo "-----" >> "$OUTPUT"

MEMORY=$(free | awk '/Mem:/ {printf "%.0f", ($3/$2)*100}')

echo "Memory Usage : ${MEMORY}%" >> "$OUTPUT"

if [ "$MEMORY" -ge "$MEMORY_THRESHOLD" ]; then

    echo " CRITICAL : Memory usage high" >> "$OUTPUT"
    CRITICAL=$((CRITICAL+1))

else

    echo " PASS : Memory normal" >> "$OUTPUT"
    PASS=$((PASS+1))

fi

# -----
# CONNECTIONS
# -----

echo "" >> "$OUTPUT"
echo "5. CONNECTIONS" >> "$OUTPUT"
echo "-----" >> "$OUTPUT"

CURRENT=$(mysql -N -e \
"SHOW GLOBAL STATUS LIKE 'Threads_connected';" | awk '{print $2}')

MAX=$(mysql -N -e \
"SHOW VARIABLES LIKE 'max_connections';" | awk '{print $2}')

if [ -n "$CURRENT" ] && [ -n "$MAX" ] && [ "$MAX" -gt 0 ]; then

    PERCENT=$((CURRENT * 100 / MAX))

    echo "Current : $CURRENT" >> "$OUTPUT"
    echo "Maximum : $MAX" >> "$OUTPUT"
    echo "Usage : ${PERCENT}%" >> "$OUTPUT"

    if [ "$PERCENT" -ge "$CONNECTION_CRITICAL" ]; then

```

```

        echo " CRITICAL : Too many connections" >> "$OUTPUT"
        CRITICAL=$((CRITICAL+1))

    elif [ "$PERCENT" -ge "$CONNECTION_WARNING" ]; then

        echo " WARNING : Connection usage high" >> "$OUTPUT"
        WARNING=$((WARNING+1))

    else

        echo " PASS : Connections normal" >> "$OUTPUT"
        PASS=$((PASS+1))

    fi

fi

# -----
# LONG RUNNING QUERIES
# -----

echo "" >> "$OUTPUT"
echo "6. LONG RUNNING QUERIES" >> "$OUTPUT"
echo "-----" >> "$OUTPUT"

LONG_QUERY=$(mysql -N -e "
SELECT COUNT(*)
FROM information_schema.PROCESSLIST
WHERE COMMAND <> 'Sleep'
AND TIME >= ${LONG_QUERY_SECONDS};
")

echo "Queries >= ${LONG_QUERY_SECONDS} sec : $LONG_QUERY" >> "$OUTPUT"

if [ "$LONG_QUERY" -gt 0 ]; then

    echo " WARNING : Long-running queries found" >> "$OUTPUT"

    mysql -e "
SELECT ID,USER,HOST,DB,TIME,STATE,
LEFT(INFO,100) AS QUERY
FROM information_schema.PROCESSLIST
WHERE COMMAND <> 'Sleep'
AND TIME >= ${LONG_QUERY_SECONDS}
ORDER BY TIME DESC;
" >> "$OUTPUT"

    WARNING=$((WARNING+1))

else

    echo " PASS : No long-running queries" >> "$OUTPUT"
    PASS=$((PASS+1))

fi

```

```

# -----
# DEADLOCKS
# -----

echo "" >> "$OUTPUT"
echo "7. DEADLOCKS" >> "$OUTPUT"
echo "-----" >> "$OUTPUT"

DEADLOCKS=$(mysql -N -e \
"SHOW GLOBAL STATUS LIKE 'Innodb_deadlocks';" | awk '{print $2}')

echo "Deadlocks : ${DEADLOCKS:-0}" >> "$OUTPUT"

if [ "${DEADLOCKS:-0}" -gt 0 ]; then

    echo "  WARNING : Deadlocks detected" >> "$OUTPUT"
    WARNING=$((WARNING+1))

else

    echo "  PASS : No deadlocks detected" >> "$OUTPUT"
    PASS=$((PASS+1))

fi

# -----
# BACKUP
# -----

echo "" >> "$OUTPUT"
echo "8. BACKUP" >> "$OUTPUT"
echo "-----" >> "$OUTPUT"

LAST_STATUS=$(find "$BACKUP_DIR" \
-name "backup.status" \
-type f \
-printf '%T@ %p\n' 2>/dev/null |
sort -n |
tail -1 |
cut -d' ' -f2-)

if [ -n "$LAST_STATUS" ]; then

    cat "$LAST_STATUS" >> "$OUTPUT"

    if grep -q "BACKUP_STATUS=SUCCESS" "$LAST_STATUS"; then

        echo "  PASS : Last backup successful" >> "$OUTPUT"
        PASS=$((PASS+1))

    else

        echo "  CRITICAL : Last backup FAILED" >> "$OUTPUT"
        CRITICAL=$((CRITICAL+1))
    fi
fi

```

```

fi

else

    echo "  CRITICAL : No backup status found" >> "$OUTPUT"
    CRITICAL=$((CRITICAL+1))

fi

# -----
# DISK
# -----

echo "" >> "$OUTPUT"
echo "9. DISK" >> "$OUTPUT"
echo "-----" >> "$OUTPUT"

df -h >> "$OUTPUT"

DISK=$(df -P /var/lib/mysql | awk 'NR==2 {gsub("%", "", $5); print $5}')

echo "" >> "$OUTPUT"
echo "MySQL Disk Usage : ${DISK}%" >> "$OUTPUT"

if [ "$DISK" -ge "$DISK_THRESHOLD" ]; then

    echo "  CRITICAL : Disk usage > ${DISK_THRESHOLD}%" >> "$OUTPUT"
    CRITICAL=$((CRITICAL+1))

else

    echo "  PASS : Disk usage normal" >> "$OUTPUT"
    PASS=$((PASS+1))

fi

# -----
# FINAL SUMMARY
# -----

echo "" >> "$OUTPUT"
echo "===== " >> "$OUTPUT"
echo "      FINAL SUMMARY" >> "$OUTPUT"
echo "===== " >> "$OUTPUT"

echo "  PASS   : $PASS" >> "$OUTPUT"
echo "  WARNING : $WARNING" >> "$OUTPUT"
echo "  CRITICAL : $CRITICAL" >> "$OUTPUT"

if [ "$CRITICAL" -gt 0 ]; then

    echo "" >> "$OUTPUT"
    echo "OVERALL STATUS :  CRITICAL" >> "$OUTPUT"

elif [ "$WARNING" -gt 0 ]; then

```



```
echo "" >> "$OUTPUT"
echo "OVERALL STATUS : WARNING" >> "$OUTPUT"
```

else

```
echo "" >> "$OUTPUT"
echo "OVERALL STATUS : HEALTHY" >> "$OUTPUT"
```

fi

```
echo "" >> "$OUTPUT"
echo "Report: $OUTPUT" >> "$OUTPUT"
```

```
cat "$OUTPUT"
```

```
sudo chmod +x /usr/local/mysql-monitor/check_mysql_health.sh
```

Test Manually :

```
/usr/local/mysql-monitor/check_mysql_health.sh
```

```
cat /usr/local/mysql-monitor/reports/$(date +%Y-%m-%d)/mysql_health_report.txt
```

```
crontab -e
```

```
*/5 * * * * /usr/local/mysql-monitor/check_mysql_health.sh >> /usr/local/mysql-  
monitor/logs/health.log 2>&1
```

```
*/5 * * * * /usr/local/mysql-monitor/check_replication.sh >> /usr/local/mysql-  
monitor/logs/replication.log 2>&1
```

```
*/10 * * * * /usr/local/mysql-monitor/check_disk.sh >> /usr/local/mysql-monitor/logs/disk.log 2>&1
```

```
0 1 * * * /usr/local/mysql-monitor/backup_mysql.sh >> /usr/local/mysql-monitor/logs/backup.log  
2>&1
```

```
/usr/local/mysql-monitor/reports/
```

```
|
```

```
└─ 2026-09-03/
```

```
|
```

```
├─ mysql_health_report.txt
```

```
├─ replication_report.txt
```

```
└─ disk_report.txt
```

```
=====
MySQL HEALTH REPORT
=====
Server : db-03
Date   : Thu Sep  3 03:55:16 PM UTC 2026

1. MYSQL SERVICE
-----
● PASS : MySQL service is running

2. MYSQL CONNECTION
-----
● PASS : MySQL connection successful

3. CPU
-----
CPU Usage : 100%
● CRITICAL : CPU > 90%

4. MEMORY
-----
Memory Usage : 10%
● PASS : Memory normal

5. CONNECTIONS
-----
Current : 9
Maximum : 151
Usage    : 5%
● PASS : Connections normal

6. LONG RUNNING QUERIES
-----
Queries >= 300 sec : 1
● WARNING : Long-running queries found
ID   USER      HOST      DB      TIME      STATE      QUERY
7    event_scheduler localhost NULL      48317      Waiting for next activation  NULL

7. DEADLOCKS
-----
Deadlocks : 0
● PASS : No deadlocks detected

8. BACKUP
-----
● CRITICAL : No backup status found

9. DISK
-----
Filesystem      Size  Used Avail Use% Mounted on
tmpfs           782M  1.3M  780M   1% /run
/dev/mapper/ubuntu--vg-ubuntu--lv 290G   19G  256G   7% /
tmpfs           3.9G   0    3.9G   0% /dev/shm
tmpfs           5.0M   0    5.0M   0% /run/lock
/dev/sdb2       2.0G  126M  1.7G   7% /boot
/dev/sdb1       1.1G   6.1M  1.1G   1% /boot/efi
tmpfs           782M  4.0K  782M   1% /run/user/0
```

CPU MONITORING SCRIPT

Add this into confi.conf file .

```
sudo vi /usr/local/mysql-monitor/config.conf
```

```
# =====
# MySQL Server Monitoring Config
# =====

SERVER_NAME="$(hostname)"
SERVER_IP="$(hostname -I | awk '{print $1}')"

CPU_THRESHOLD=90
```

```
# CPU must remain above threshold for 5 checks
CONSECUTIVE_CHECKS=5

# CPU must fall below this value to consider recovered
RECOVERY_THRESHOLD=80

# Email
ALERT_EMAIL="dba@example.com"

# Monitoring directories
BASE_DIR="/usr/local/mysql-monitor"
LOG_DIR="${BASE_DIR}/logs"
REPORT_DIR="${BASE_DIR}/reports"

CPU_LOG="${LOG_DIR}/cpu_monitor.log"
CPU_STATE="${LOG_DIR}/cpu_alert.state"
```

Cpu monitoring script

```
sudo vi /usr/local/mysql-monitor/cpu_monitor.sh

#!/bin/bash

# =====
# CPU Monitoring + Email Alert
# =====

source /usr/local/mysql-monitor/config.conf

mkdir -p "$LOG_DIR"
mkdir -p "$REPORT_DIR"

DATE=$(date '+%Y-%m-%d %H:%M:%S')

# -----
# Get CPU utilization
# -----

CPU_USAGE=$(top -bn1 | awk '/Cpu\(s\) / {
    printf "%.0f", 100 - $8
}')

# -----
# Validate CPU value
# -----

if ! [[ "$CPU_USAGE" =~ ^[0-9]+$ ]]; then
    echo "$DATE ERROR: Unable to determine CPU usage" >> "$CPU_LOG"
    exit 1
fi

echo "$DATE CPU=${CPU_USAGE}%" >> "$CPU_LOG"

# -----
# Read previous state
#
```

```

# State format:
# HIGH_COUNT|ALERT_SENT
#
# Example:
# 3|0
# -----

if [ -f "$CPU_STATE" ]; then
    HIGH_COUNT=$(cut -d'|' -f1 "$CPU_STATE")
    ALERT_SENT=$(cut -d'|' -f2 "$CPU_STATE")
else
    HIGH_COUNT=0
    ALERT_SENT=0
fi

# -----
# CPU HIGH
# -----

if [ "$CPU_USAGE" -ge "$CPU_THRESHOLD" ]; then

    HIGH_COUNT=$((HIGH_COUNT + 1))

    echo "$DATE WARNING: CPU=${CPU_USAGE}% HIGH_COUNT=${HIGH_COUNT}" \
        >> "$CPU_LOG"

    # -----
    # Send alert after consecutive high checks
    # -----

    if [ "$HIGH_COUNT" -ge "$CONSECUTIVE_CHECKS" ] && [ "$ALERT_SENT" -eq 0 ]; then

        SUBJECT="CRITICAL: ${SERVER_NAME} CPU High - ${CPU_USAGE}%"

        BODY="
MYSQL SERVER CPU ALERT
=====

Server    : ${SERVER_NAME}
IP Address : ${SERVER_IP}

CPU Usage  : ${CPU_USAGE}%
Threshold  : ${CPU_THRESHOLD}%

High CPU Duration:
${CONSECUTIVE_CHECKS} consecutive checks

Time:
${DATE}

Status:
CRITICAL

Please investigate the server immediately.

Useful commands:

```

```

top
uptime
free -h
iostat
mysql -e 'SHOW FULL PROCESSLIST;'
"

    echo "$BODY" | mail -s "$SUBJECT" "$ALERT_EMAIL"

    if [ $? -eq 0 ]; then
        echo "$DATE ALERT EMAIL SENT" >> "$CPU_LOG"
        ALERT_SENT=1
    else
        echo "$DATE ERROR: Failed to send alert email" >> "$CPU_LOG"
    fi

fi

# -----
# CPU NORMAL / RECOVERY
# -----

elif [ "$CPU_USAGE" -lt "$RECOVERY_THRESHOLD" ]; then

    echo "$DATE INFO: CPU recovered to ${CPU_USAGE}%" \
    >> "$CPU_LOG"

    # -----
    # Send recovery email only if alert was previously sent
    # -----

    if [ "$ALERT_SENT" -eq 1 ]; then

        SUBJECT="RECOVERY: ${SERVER_NAME} CPU Normal - ${CPU_USAGE}%"

        BODY="
MYSQL SERVER CPU RECOVERY
=====

Server    : ${SERVER_NAME}
IP Address : ${SERVER_IP}

Current CPU : ${CPU_USAGE}%
Alert Level : ${CPU_THRESHOLD}%

Status:
RECOVERED

Time:
${DATE}

CPU has returned to normal.
"

        echo "$BODY" | mail -s "$SUBJECT" "$ALERT_EMAIL"

        if [ $? -eq 0 ]; then

```

```

        echo "$DATE RECOVERY EMAIL SENT" >> "$CPU_LOG"
    else
        echo "$DATE ERROR: Failed to send recovery email" >> "$CPU_LOG"
    fi
fi

# Reset
HIGH_COUNT=0
ALERT_SENT=0

fi

# -----
# Save state
# -----

echo "${HIGH_COUNT}|${ALERT_SENT}" > "$CPU_STATE"

exit 0

```

sudo chmod +x /usr/local/mysql-monitor/cpu_monitor.sh

Test manually .

/usr/local/mysql-monitor/cpu_monitor.sh

Check log.

cat /usr/local/mysql-monitor/logs/cpu_monitor.log

cat /usr/local/mysql-monitor/logs/cpu_alert.state

Install mail .

RHEL/CentOS/Rocky/AlmaLinux

sudo dnf install mailx -y

Older system : sudo yum install mailx -y

Ubuntu :

sudo apt install mailutils -y

Sample test :

```

echo "Test email from MySQL monitoring server" | mail \
-s "MySQL Monitoring Test" \
dba@example.com

```

Crontab :

Crontab -e

```

* * * * * /usr/local/mysql-monitor/cpu_monitor.sh

```

Every 1 minute

↓

cpu_monitor.sh

↓

CPU check